Patrones

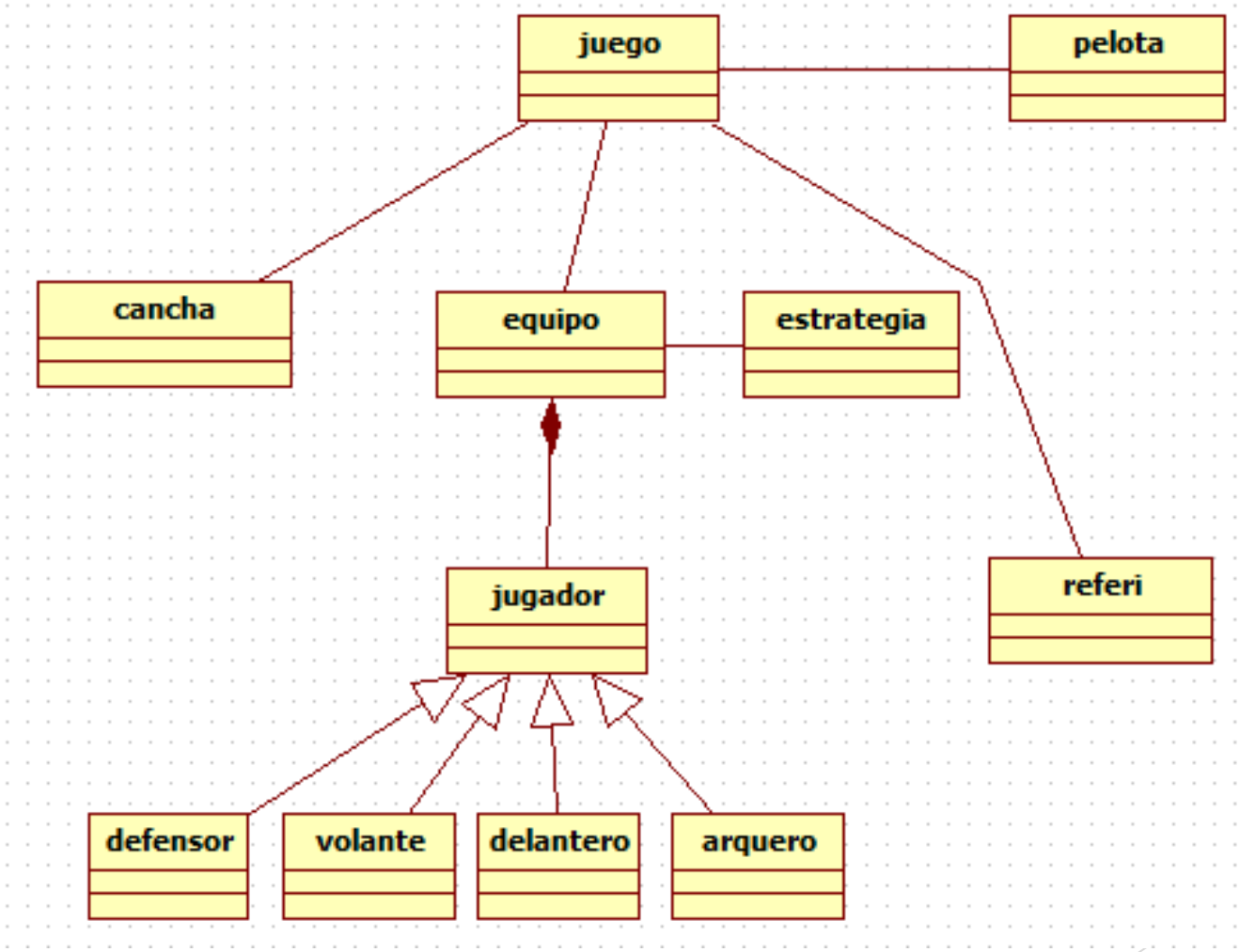
EJERCICIO: FUTBOL

- 1) Asumimos que el usuario opera las siguientes funcionalidades
  - Inicio de juego
  - Selección de dos equipos
  - Agregar – eliminar jugadores
  - Seleccionar la cancha.

Identificar los objetos

Listar los objetos mas importantes
Hacer un modelo de Dominio, simple

Identificar los objetos



Problema a resolver

- ***Problema Especifico de Diseño:*** *"Cuando la posición de la pelota cambia , todos los jugadores deben ser notificados."*

- Categoría de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

- ***Problema Generalizados: "……"***

Problema a resolver

- ***Problema Especifico de Diseño:*** *“Cuando la posicion de la pelota cambia , todos los jugadores deben ser notificados."*

- Categoría de Patrones: Comportamiento

- ***Problema Generalizados:*** *“Cuando un objeto cambia todos los objetos dependientes, deben ser notificados"*

Patrones Comportamiento



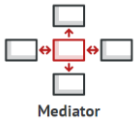
- Chain of Responsibility (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.



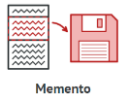
- Command (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- Interpreter (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.



- Iterator (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.



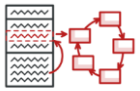
- Mediator (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.



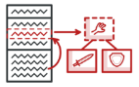
- Memento (Recuerdo): Permite volver a estados anteriores del sistema.



- Observer (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.



- State (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.



- Strategy (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.

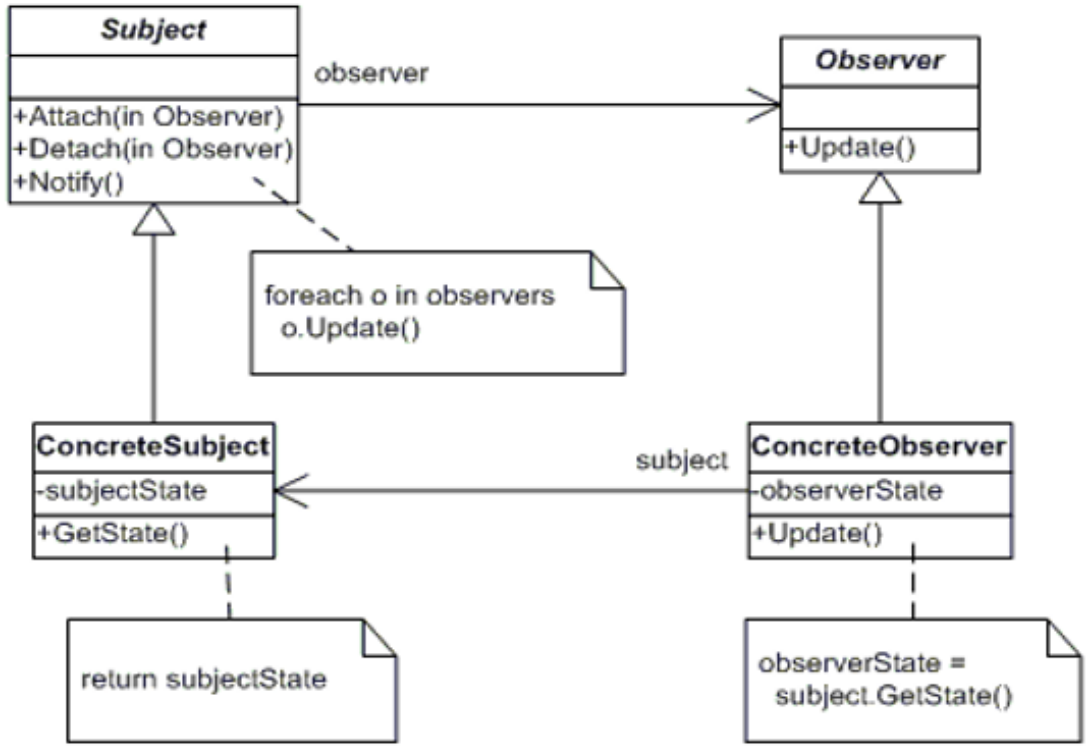


- Template Method (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.



- Visitor (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

Observer



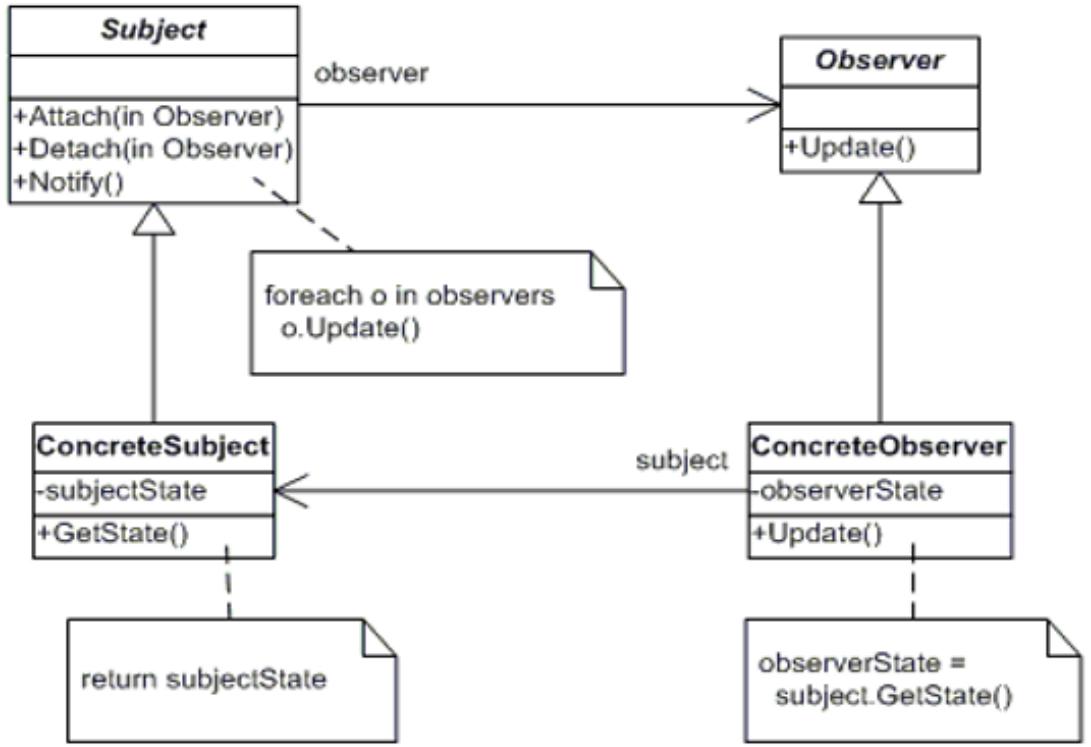
Subject:conoce sus observadores y proporciona una interfaz para gestionarlos

ConcreteSubject: almacena el estado de interés y envía una notificación a sus observadores cuando su estado cambia

Observer: define una interfaz para los objetos a los que debe notificarse el cambio de estado en ConcreteSubject

ConcretObserver: mantiene una referencia a un objeto ConcreteSubject, mantiene información consistente relacionada con el estado implementando el método Update()

Observer



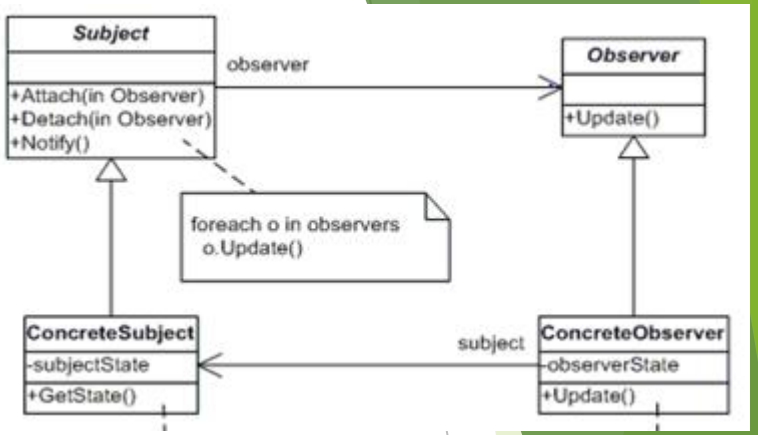
Subject:conoce sus observadores y proporciona una interfaz para gestionarlos

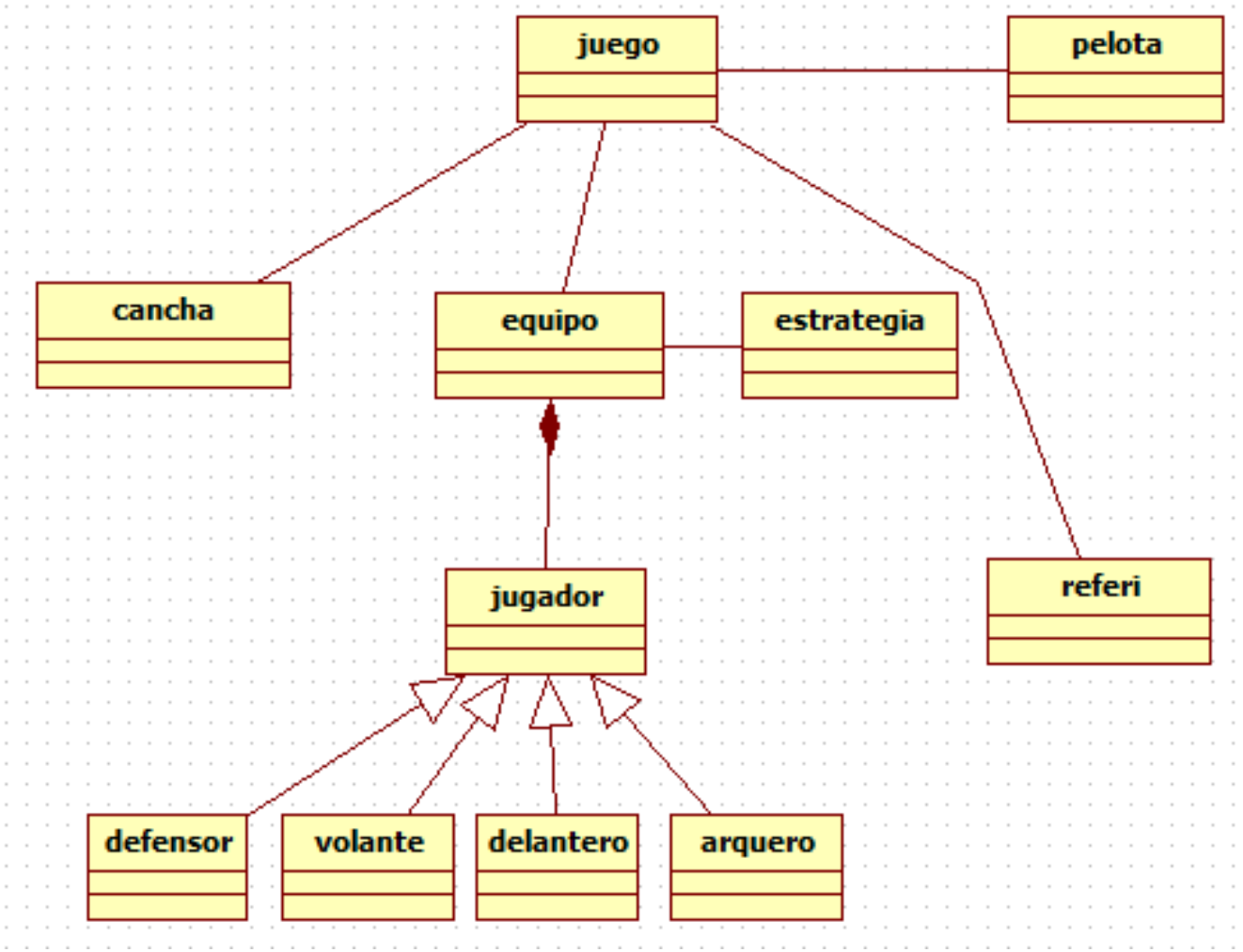
ConcreteSubject: almacena el estado de interés y envía una notificación a sus observadores cuando su estado cambia

Observer: define una interfaz para los objetos a los que debe notificarse el cambio de estado en ConcreteSubject

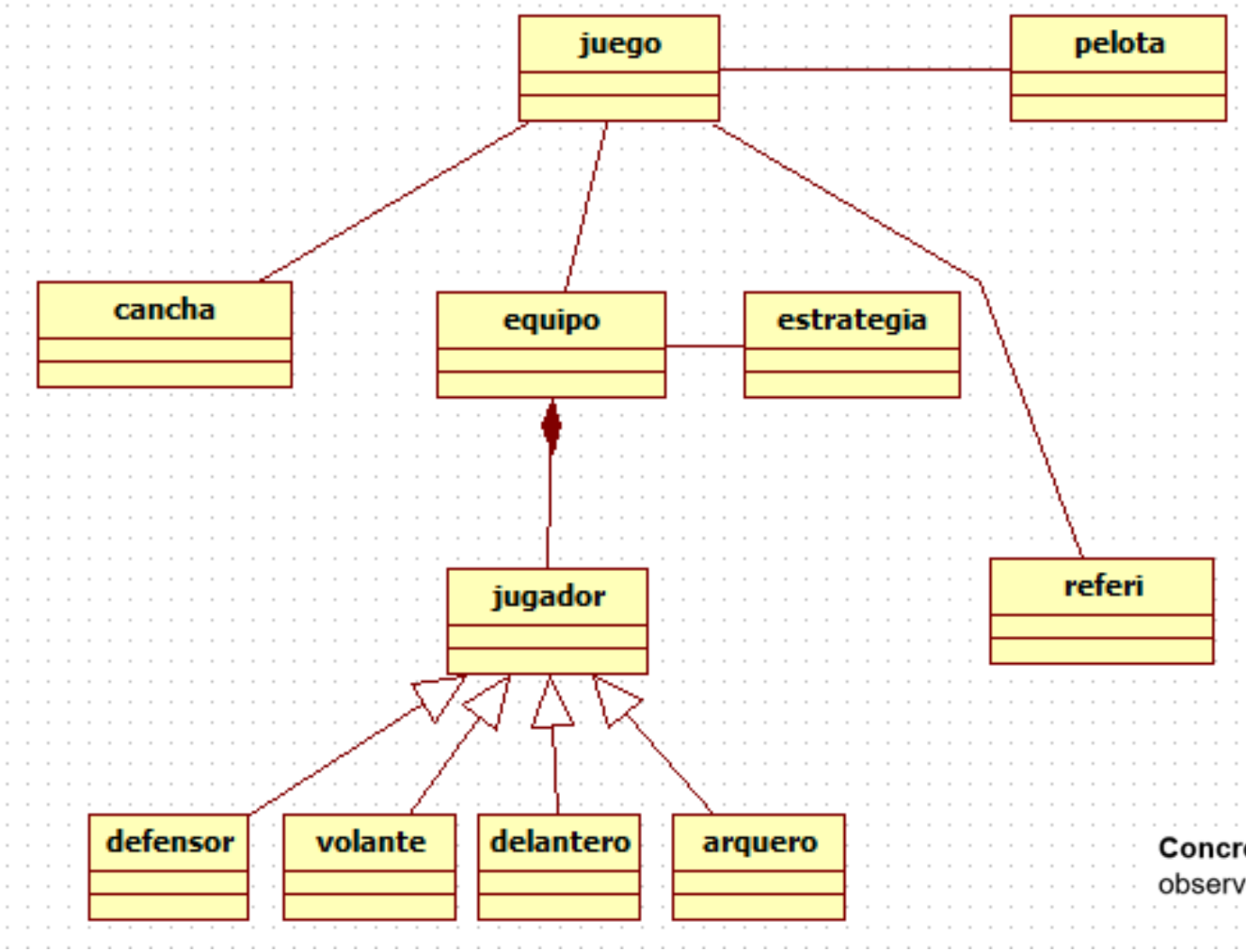
ConcretObserver: mantiene una referencia a un objeto ConcreteSubject, mantiene información consistente relacionada con el estado implementando el método Update()

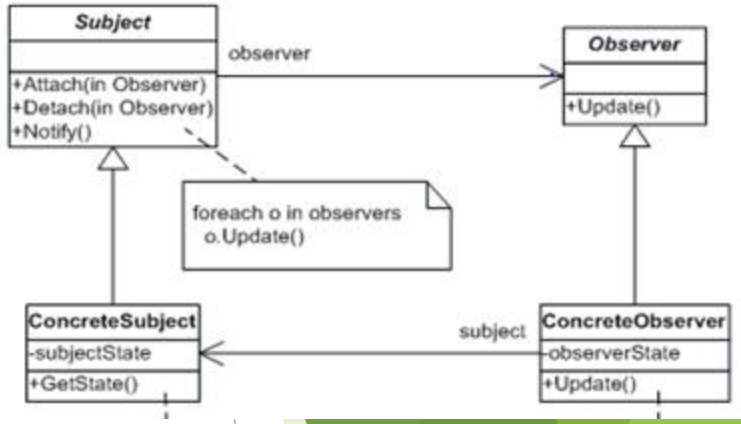
Cual es el objeto Observado?
Cual es el objeto que observa?





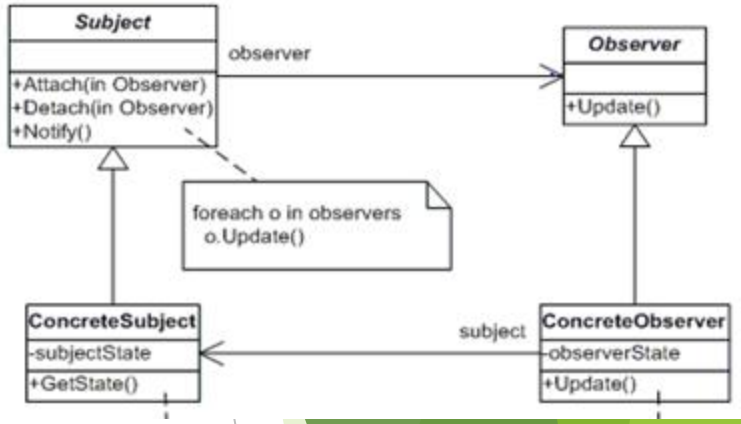
Cual es el objeto Observado?

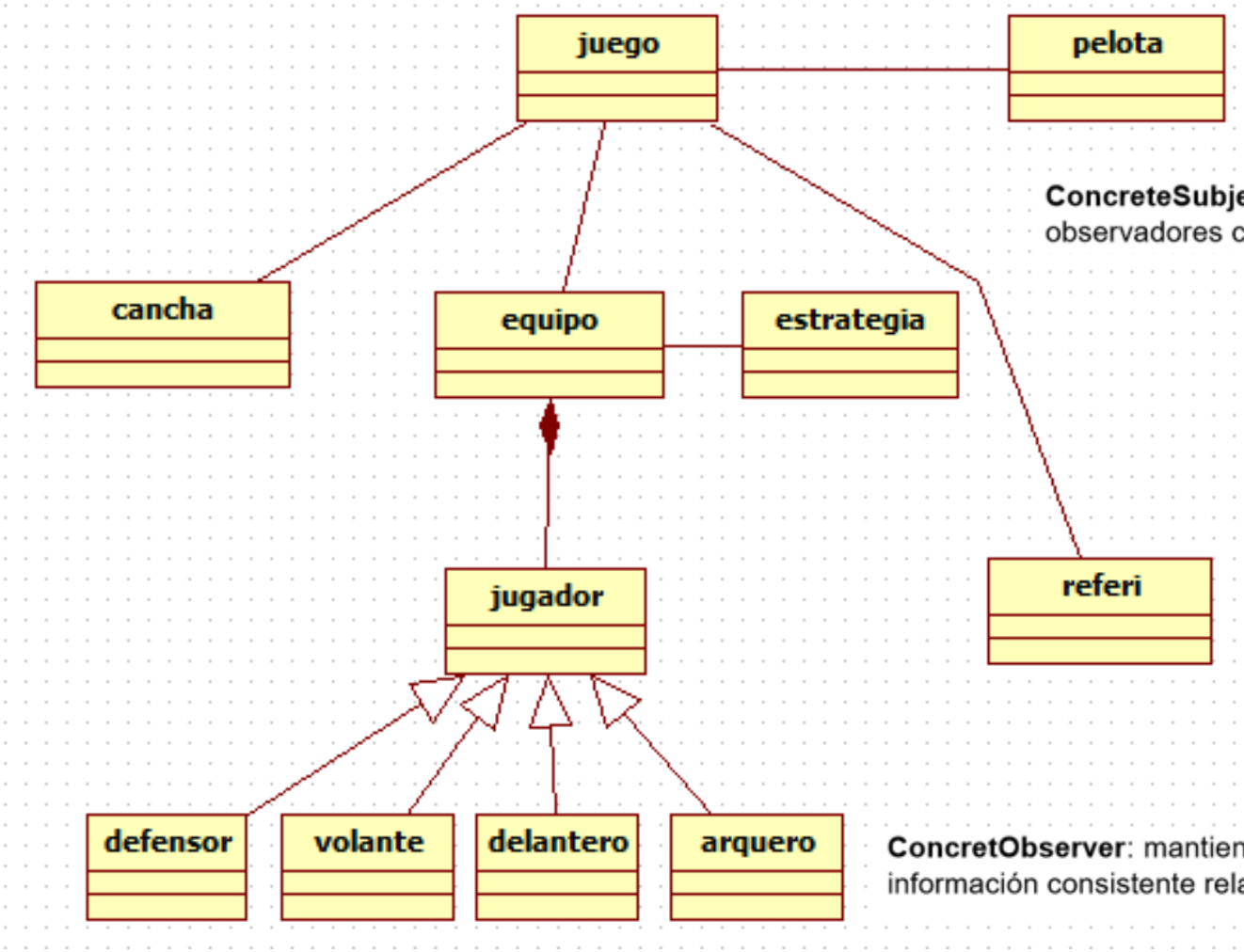




ConcreteSubject: almacena el estado de interés y envía una notificación a s
observadores cuando su estado cambia

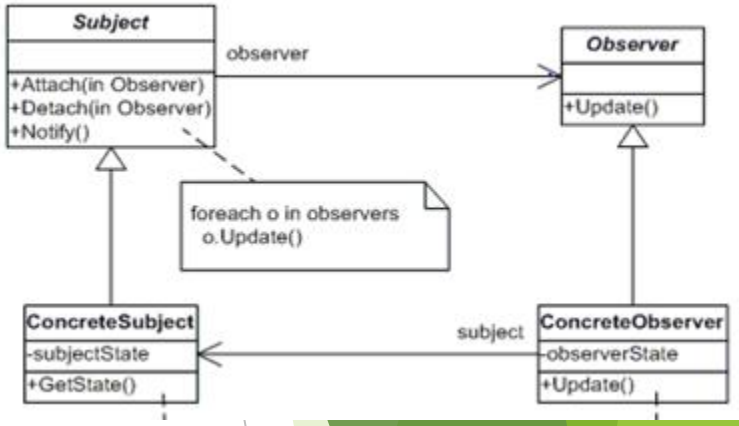
Cual es el objeto que observa?

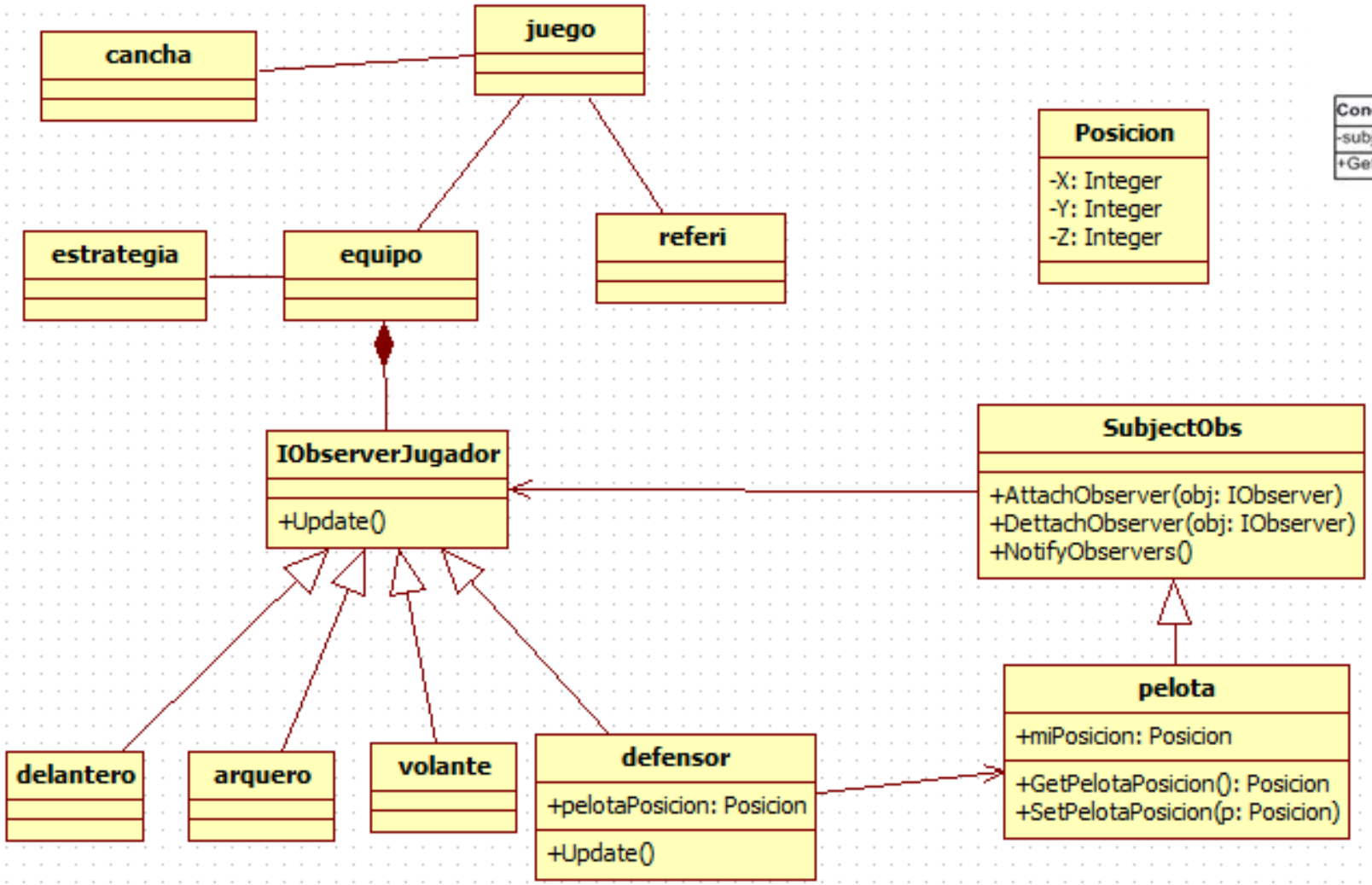




ConcreteSubject: almacena el estado de interés y envía una notificación a sus observadores cuando su estado cambia

ConcretObserver: mantiene una referencia a un objeto ConcreteSubject, mantiene información consistente relacionada con el estado implementando el método Update()


Subject
+Attach(in Observer)
+Detach(in Observer)
+Notify()
observer
Observer
+Update()
foreach o in observers
o.Update()
ConcreteSubject
-subjectState
+GetState()
subject
ConcreteObserver
-observerState
+Update()


cancha
juego
estrategia
equipo
referi
Posicion
-X: Integer
-Y: Integer
-Z: Integer
IObserverJugador
+Update()
SubjectObs
+AttachObserver(obj: IObserver)
+DettachObserver(obj: IObserver)
+NotifyObservers()
delantero
arquero
volante
defensor
+pelotaPosicion: Posicion
+Update()
pelota
+miPosicion: Posicion
+GetPelotaPosicion(): Posicion
+SetPelotaPosicion(p: Posicion)

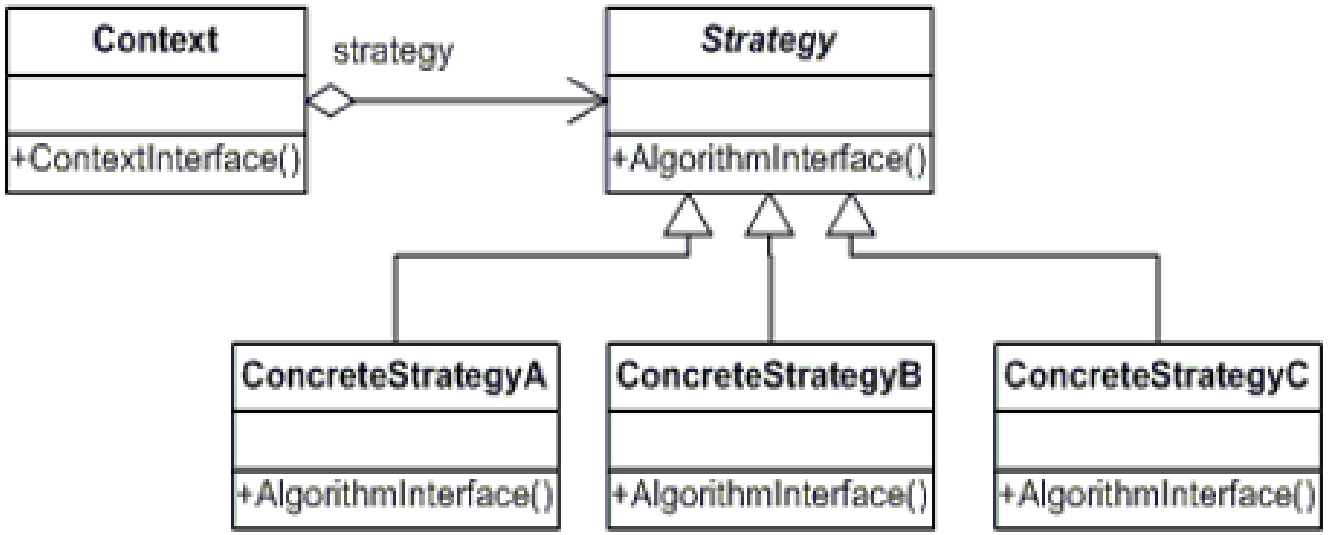
Problema a resolver

1. ***Problema Especifico de Diseño:*** *"cuando el juego esta en progreso, el usuario puede cambiar la estrategia de su equipo"*

- Categoria de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

- ***Problema Generalizados:*** *"……"*

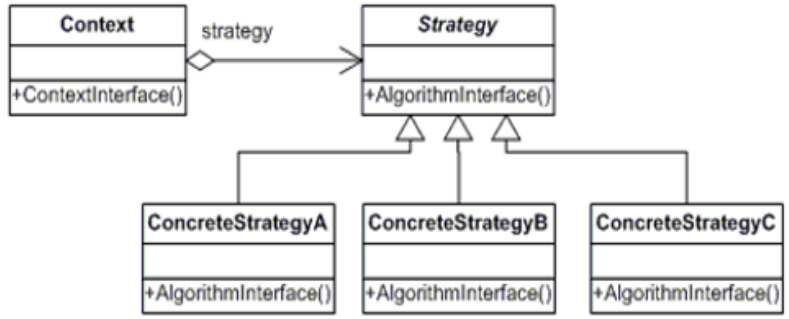
Strategy

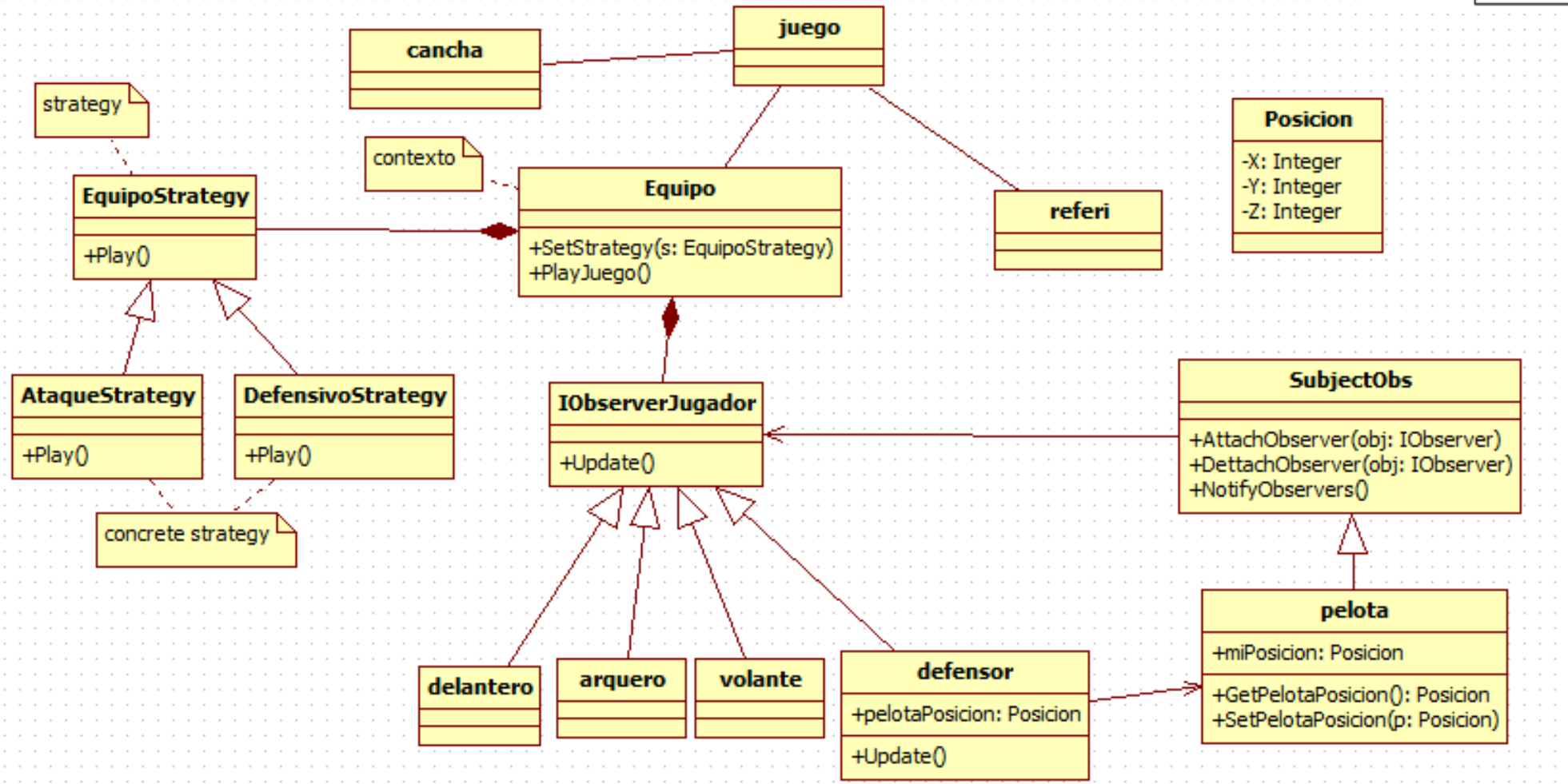


Strategy:declara un interfaz común para dar soporte a todos los algoritmos. Context utiliza esta interfaz para llamar al algoritmo definido por un ConcreteStrategy

ConcreteStrategy: Implementa el algoritmo usando la interfaz Strategy

Context:se configura con un objeto ConcreteStrategy, sobre el que mantiene una referencia, puede definir una interfaz que permita a Strategy acceder a sus datos.


Context
strategy
Strategy
+ContextInterface()
+AlgorithmInterface()
ConcreteStrategyA
ConcreteStrategyB
ConcreteStrategyC
+AlgorithmInterface()
+AlgorithmInterface()
+AlgorithmInterface()

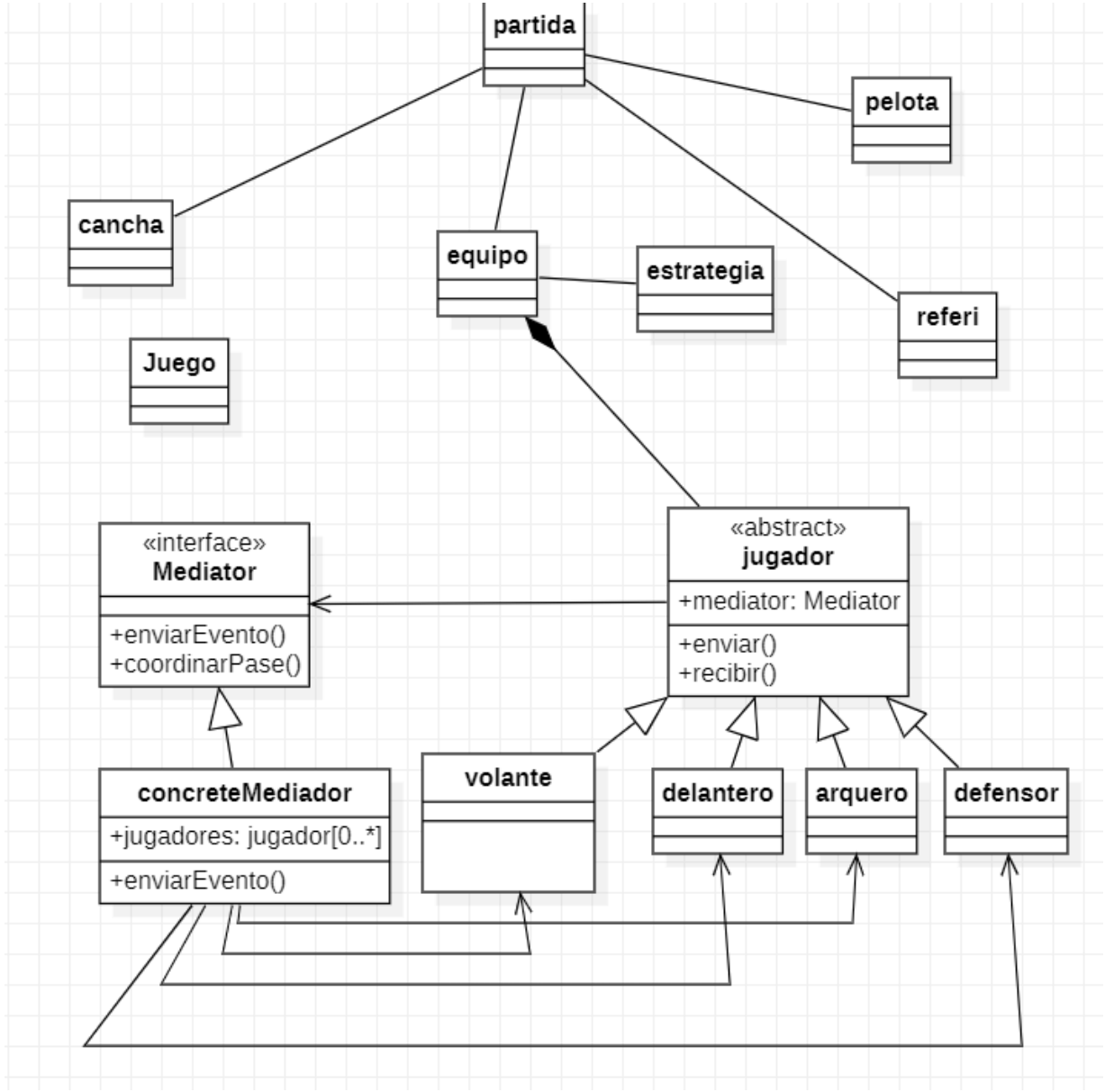

cancha
juego
strategy
contexto
EquipoStrategy
+Play()
Equipo
+SetStrategy(s: EquipoStrategy)
+PlayJuego()
referi
Posicion
-X: Integer
-Y: Integer
-Z: Integer
AtaqueStrategy
+Play()
DefensivoStrategy
+Play()
concrete strategy
IObserverJugador
+Update()
SubjectObs
+AttachObserver(obj: IObserver)
+DettachObserver(obj: IObserver)
+NotifyObservers()
delantero
arquero
volante
defensor
+pelotaPosicion: Posicion
+Update()
pelota
+miPosicion: Posicion
+GetPelotaPosicion(): Posicion
+SetPelotaPosicion(p: Posicion)

Problema a resolver

1. ***Problema Especifico de Diseño:*** *"Los jugadores se* ***comunican*** *demasiados entre sí, todos envían mensajes o estados para ver como cada uno debe jugar, por ejemplo , el* delantero habla con volante , el volante habla con arquero , el defensor con árbitro"

- Categoria de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

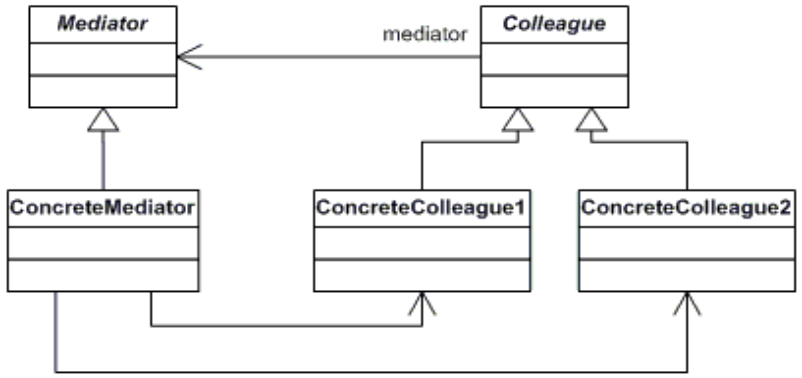
Mediator



Mediator:define una interfaz para la comunicación entre objetos Colleagues

ConcreteMediator:Implement a un comportamiento cooperativo coordinando objetos Colleagues

Colleague classes:cada objeto Colleague conoce su objeto Mediator, cada colleague comunica con su mediador cuando tiene que comunicar con otro colleague



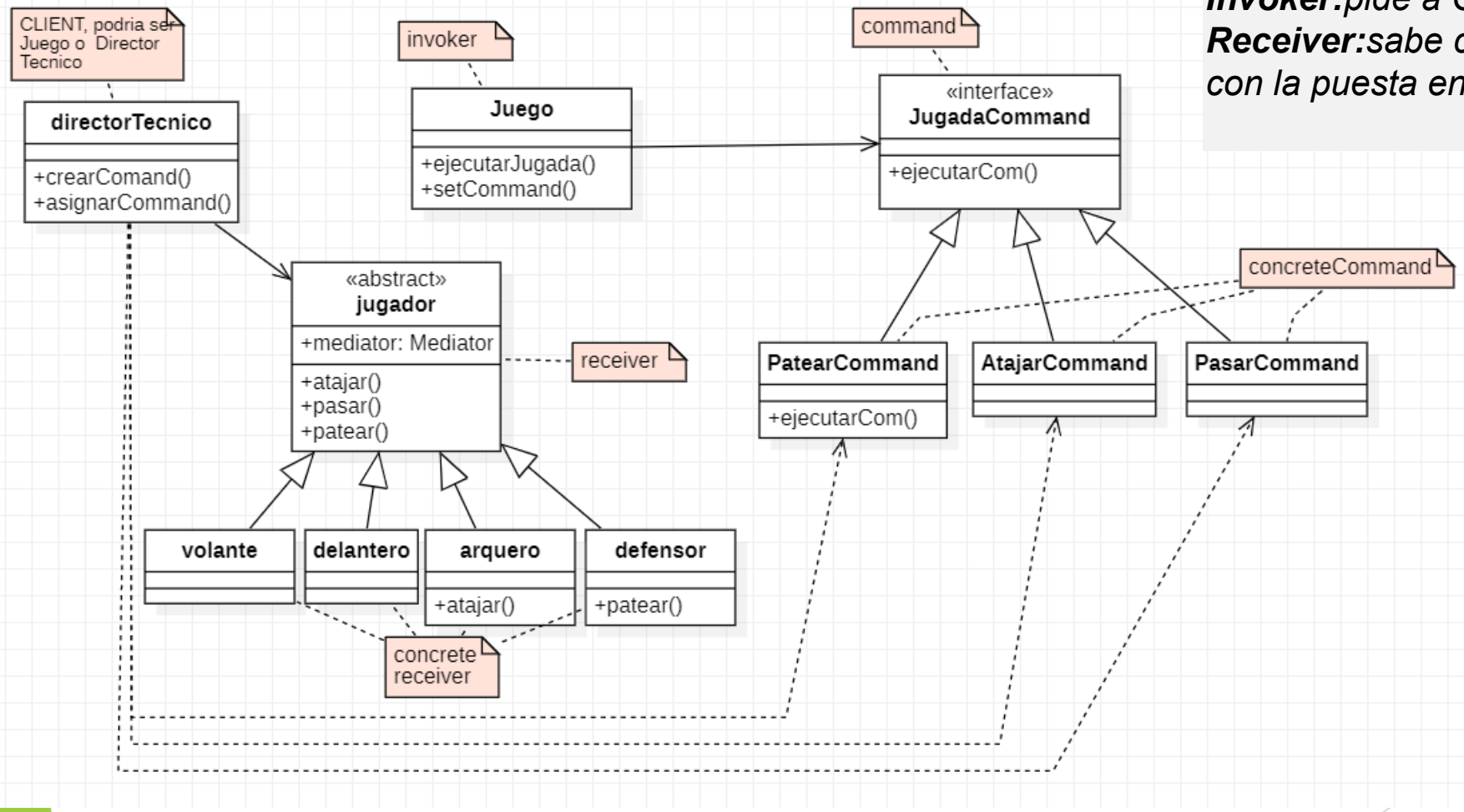
Problema a resolver

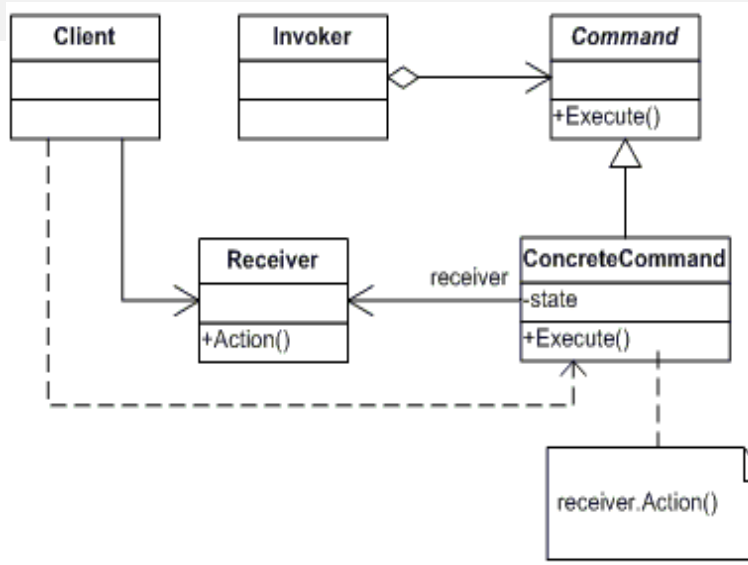
1. ***Problema Especifico de Diseño**: "cada jugador tiene muchas acciones en el juego: patear, saltar, pasar, correr, atajar, etc. cada clase tiene mucho comportamiento, se pide encapsular las acciones del jugador."*

- Categoría de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?
- Command (patrón de comportamiento) que transforma una solicitud o acción en un objeto independiente.

Command

Command: *declara un interface para la ejecución de una operación*
ConcreteCommand: *define un vínculo entre un objeto Receiver y una acción, implementa Execute() invocando al método correspondiente de Receiver*
Cliente:*Crea un objeto ConcreteCommand y establece su receptor*
Invoker:*pide a Command que lleve a cabo su petición*
Receiver:*sabe cómo realizar las operaciones asociadas con la puesta en marcha de la petición*





Command

Ventajas y Desventajas

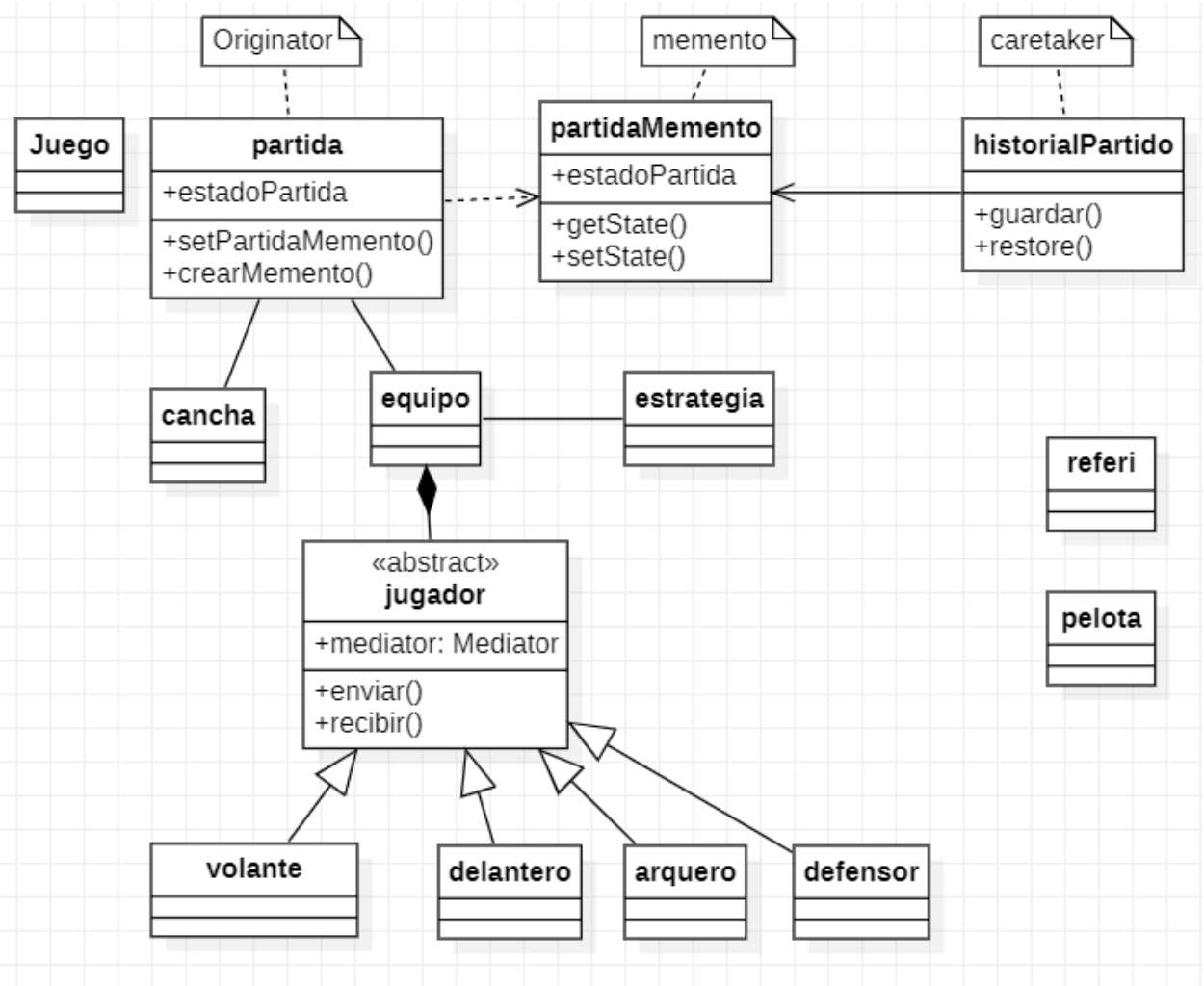
Ventajas	Desventajas
Desacoplamiento total: El emisor y el receptor no se conocen entre sí.	**Mayor número de clases**: Cada acción individual requiere crear una nueva clase concreta.
Código extensible: Puedes añadir nuevos comandos sin romper el código existente (Principio Abierto/Cerrado).	**Complejidad de diseño**: Introduce capas adicionales entre los emisores y los receptores de la lógica.
Comandos compuestos: Permite agrupar varios comandos en uno solo (*Macro Commands*).	**Gestión de estado**: Guardar el estado histórico para deshacer puede volverse complejo.

Problema a resolver

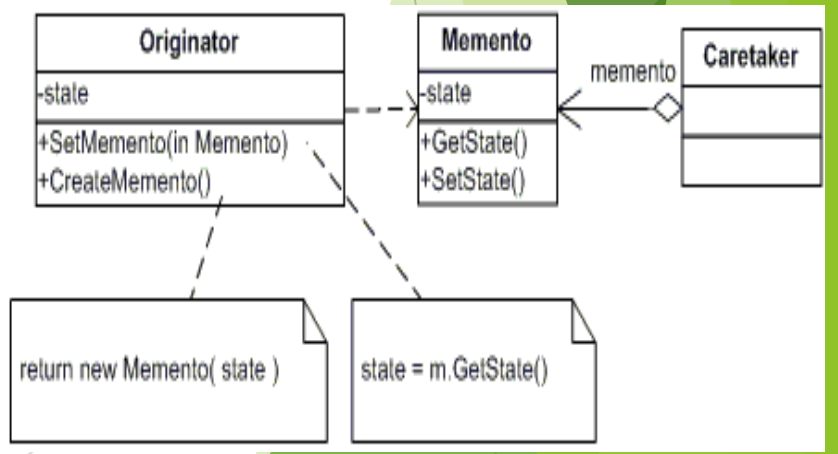
1. ***Problema Especifico de Diseño**: "Queremos guardar estado del juego/partido: marcador, posiciones, tiempo, etc. para restaurarlo."*

- Categoria de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

Memento

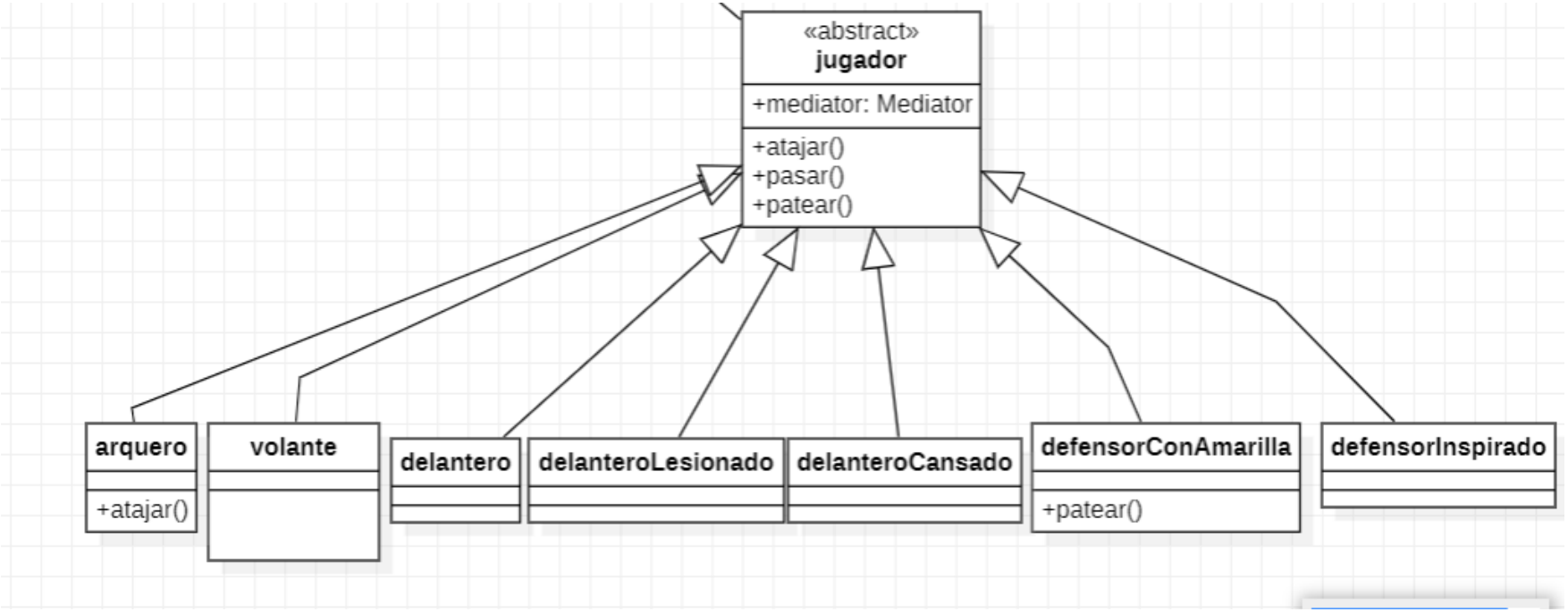


***Memento**:Almacena el estado interno del objeto Originator, protege del acceso por parte de objetos distintos al Originator*
***Originator:**crea un objeto memento, que contiene una foto fija de su estado interno*
***Caretaker:**es el responsable de mantener la seguridad del objeto memento, no opera o examina el contenido de memento*



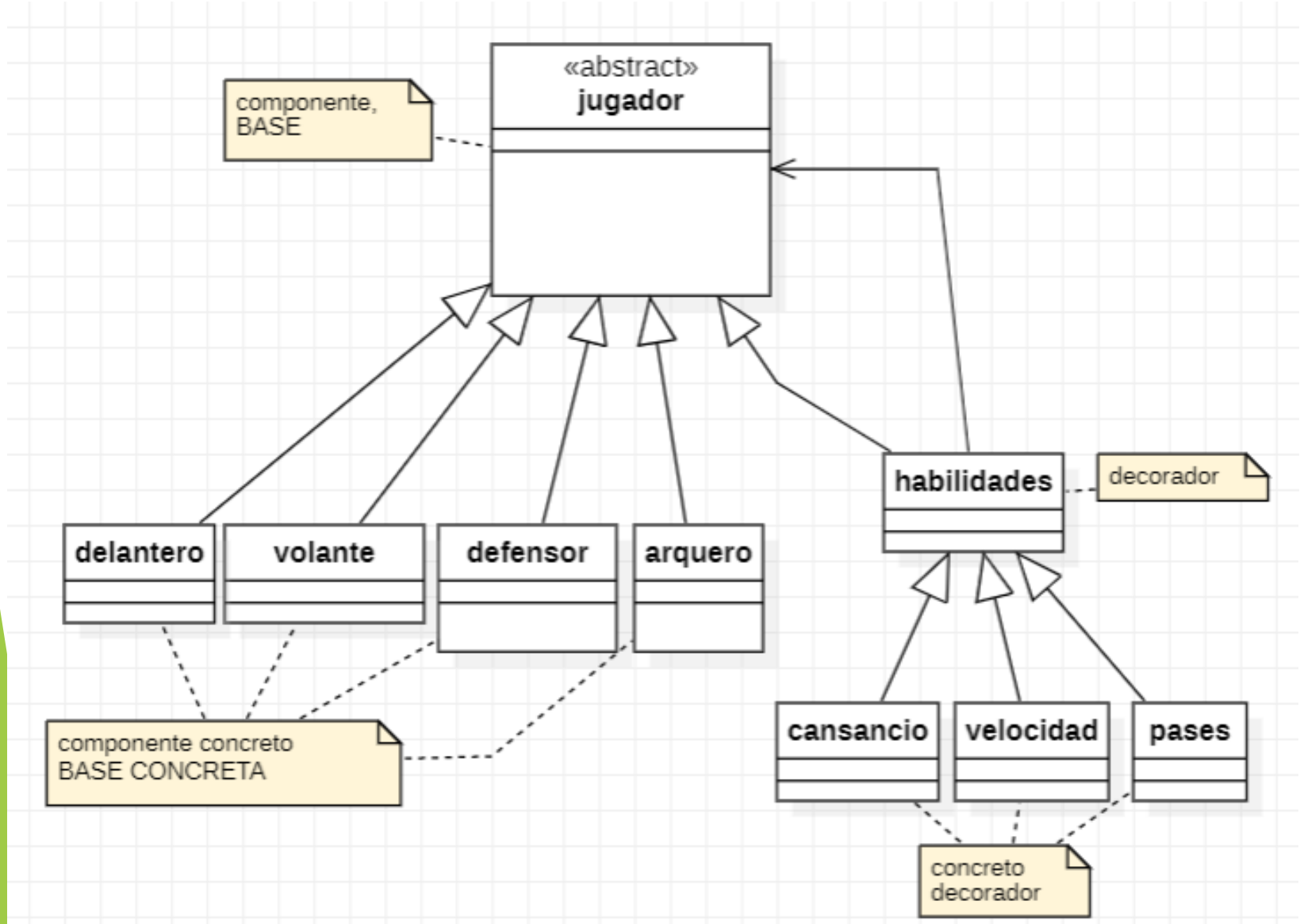
Problema a resolver

1. ***Problema Especifico de Diseño**: "Queremos agregar habilidades o estados especiales a jugadores, ejemplo cansado, lesionado, inspirado, con tarjeta amarilla, velocidad extra, SIN crear muchas subclases."*



- Categoría de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

Decorator

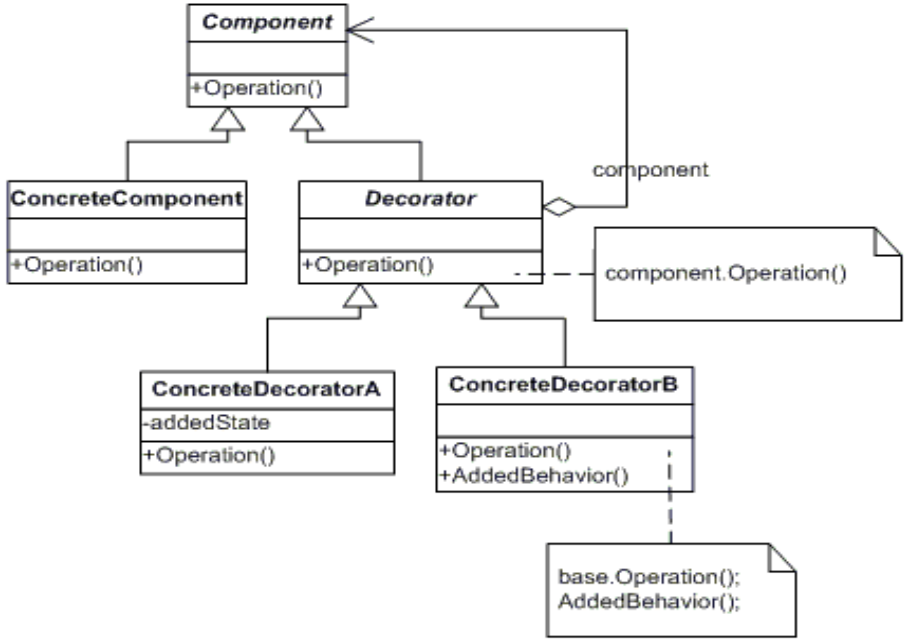


Component: Define la interfaz que ofrecen los objetos que poseen responsabilidades añadidas dinámicamente

ConcreteComponent:Define un objeto al que responsabilidades adicionales pueden serle añadidas

Decorator: Mantiene referencia a un objeto Component y define una interfaz que conforma el interfaz del Component

ConcreteDecorator:Añade las responsabilidades al Component



Problema a resolver

1. ***Problema Especifico de Diseño:*** *"En un estadio hay miles de hinchas. Muchos comparten: equipo, tipo de camiseta, color, canción, sector, comportamiento, etc, Si cada objeto Hincha guarda toda esa información repetida, tenemos un problema de estructura."*

- Categoría de Patrones?
  - Resuelve un problema de Creación ?
  - Resuelve un problema de Estructura?
  - Resuelve un problema de Comportamiento?

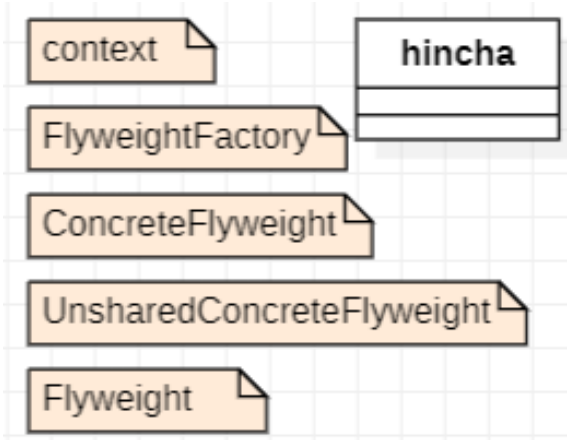
Flyweight

Estado intrínseco (compartido). Información igual para muchos hinchas/aficionados:

- Camiseta - colores
- Cantos - escudo
- tipo de hincha

Estado extrínseco (único). Información propia de cada hincha/aficionados:

- Nombre - edad
- Asiento - posición en tribuna



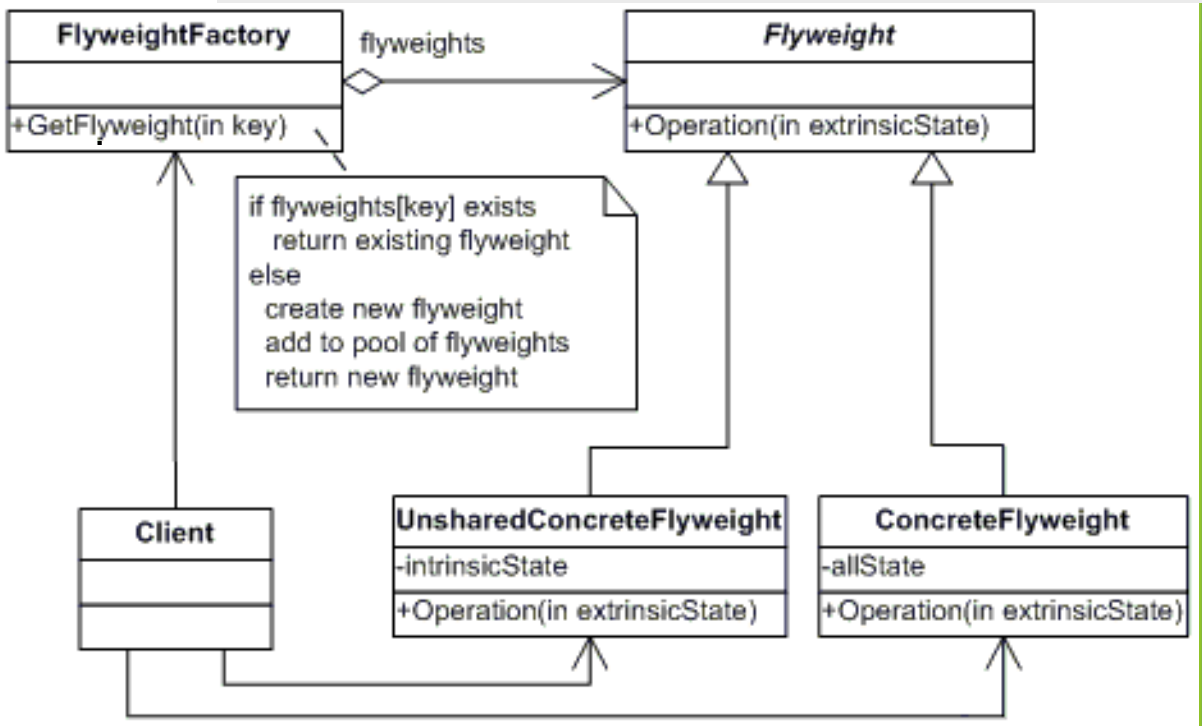
Flyweight:declaran una interface a través de la que flyweights pueden recibir y actuar sobre estados externos

ConcreteFlyweight:implementa la interfaz del flyweight y añade almacenamiento para el estado interno

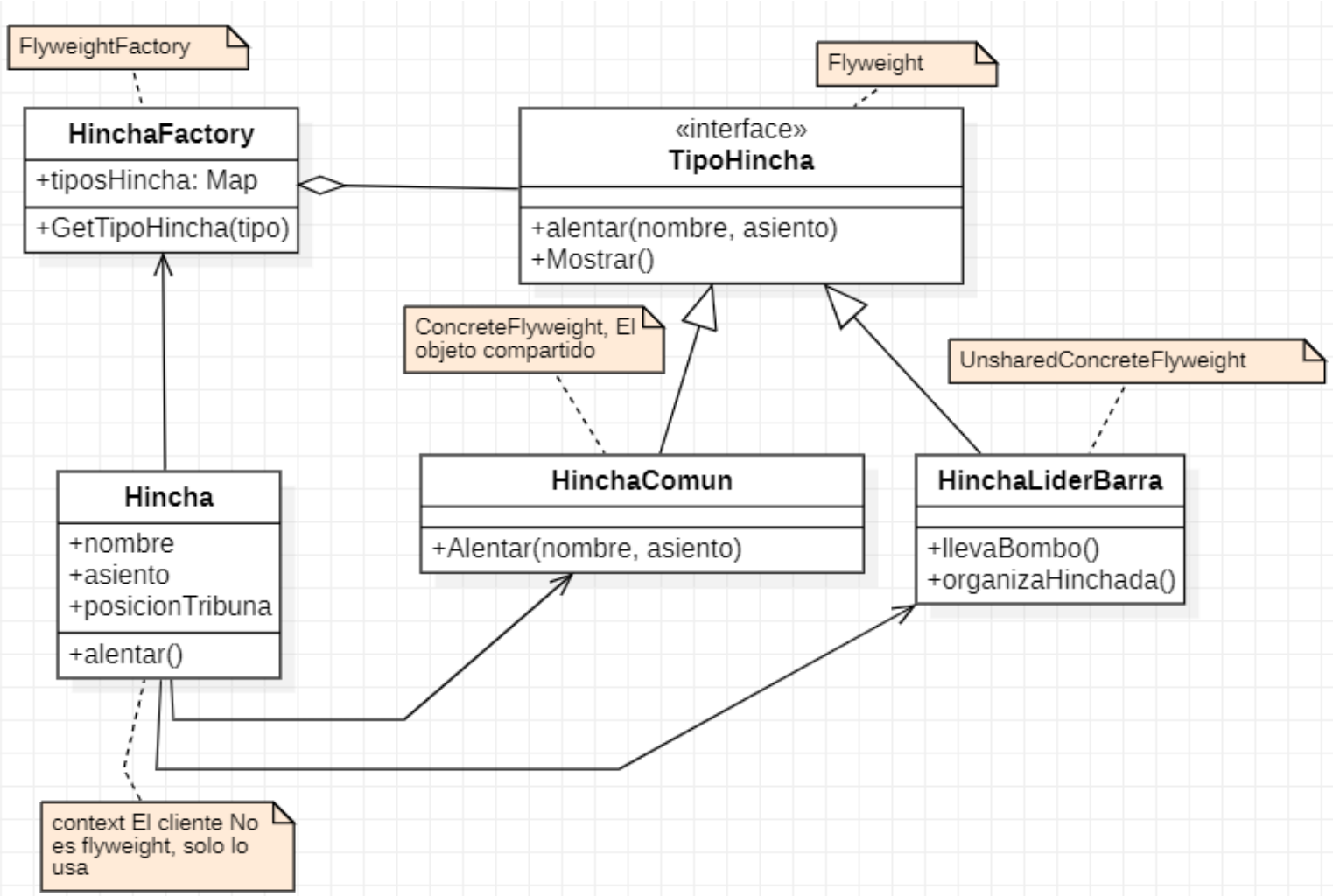
UnsharedConcreteFlyweight:no todas las subclases de Flyweight necesitan ser compartidas

FlyweightFactory:crea y gestiona los objetos Flyweight.

Client: mantiene una referencia a objetos flyweights



Flyweight



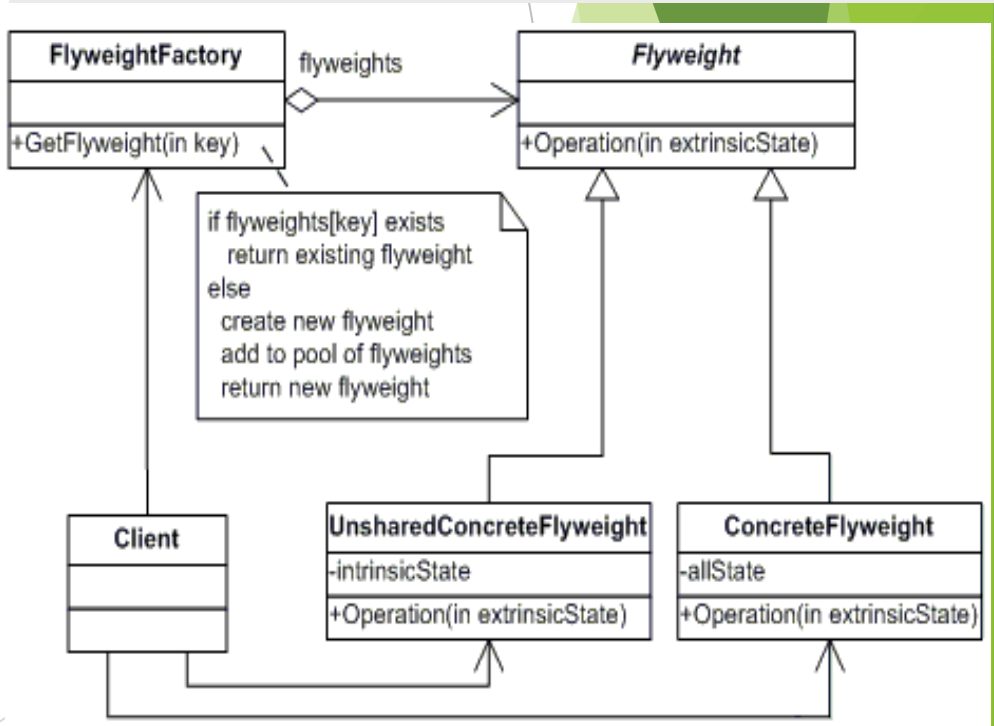
Flyweight:declaran una interface a través de la que flyweights pueden recibir y actuar sobre estados externos

ConcreteFlyweight:implementa la interfaz del flyweight y añade almacenamiento para el estado interno

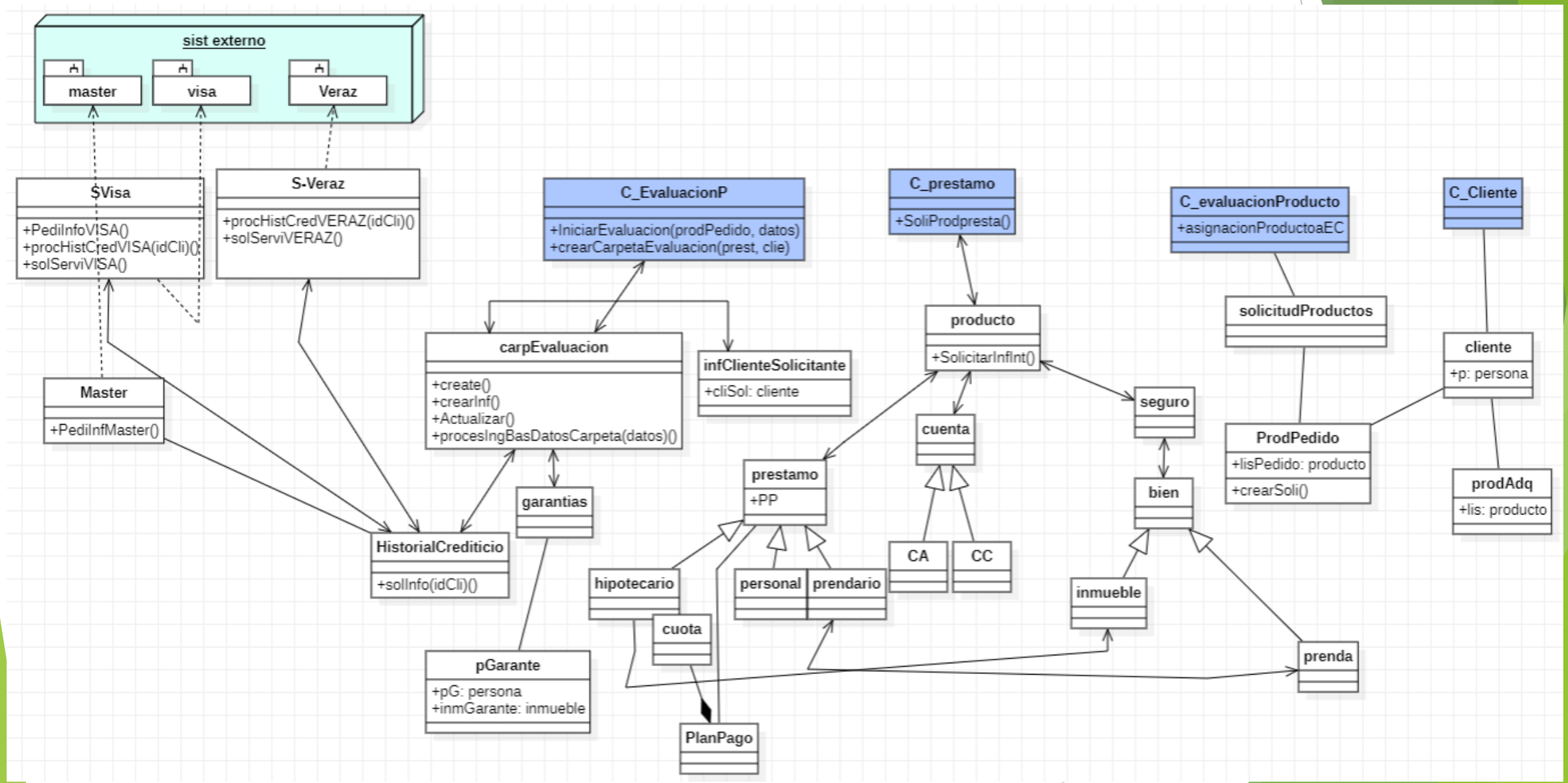
UnsharedConcreteFlyweight:no todas las subclases de Flyweight necesitan ser compartidas

FlyweightFactory:crea y gestiona los objetos Flyweight.

Client: mantiene una referencia a objetos flyweights



EJERCICIO: CASO BANCO OLIVARES



EJERCICIO: CASO BANCO OLIVARES

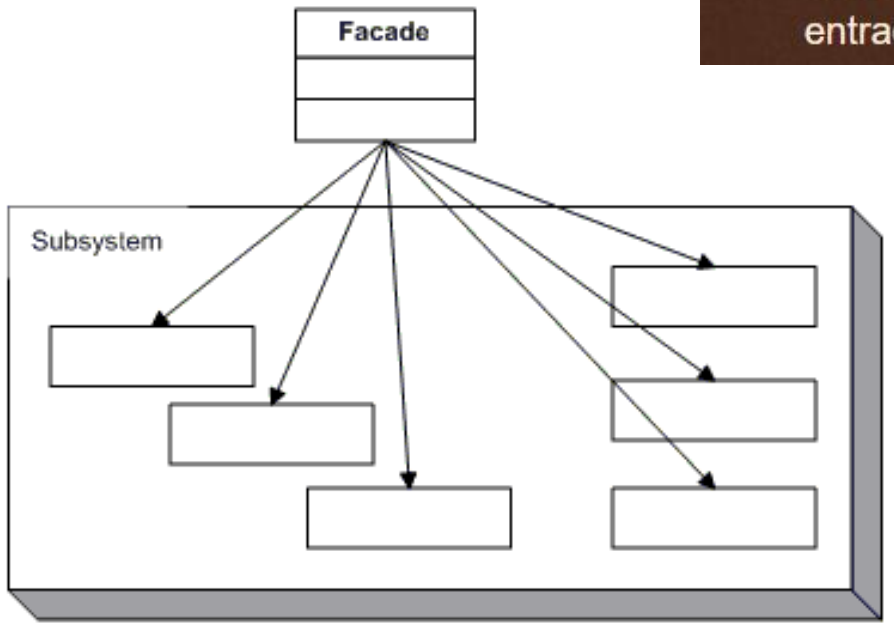
- ***Problema Especifico de Diseño**: "hay que interactuar con otros sistemas."*
- Categoria de Patrones: Estructura
- ***Solución**: "Crear una Clase que proporcione una capa de abstracción que permite a los clientes interactuar con el sistema a través de una interfaz más sencilla y controlada, ocultando las complejidades internas*

Facade



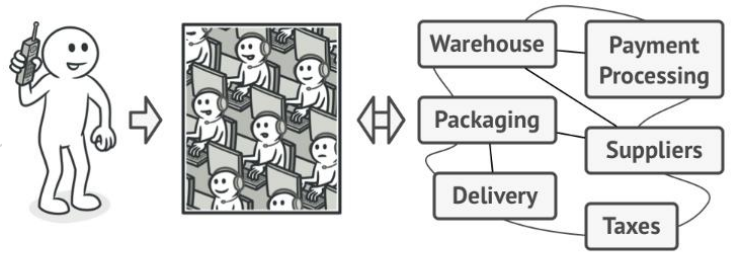
Se debe utilizar cuando:

- Se quiera proporcionar una interfaz sencilla para un subsistema complejo.
- Se quiera desacoplar un subsistema de sus clientes y de otros subsistemas, haciéndolo más independiente y portable.
- Se quiera dividir los sistemas en niveles: las fachadas serían el punto de entrada a cada nivel. Facade puede ser utilizado a nivel aplicación.



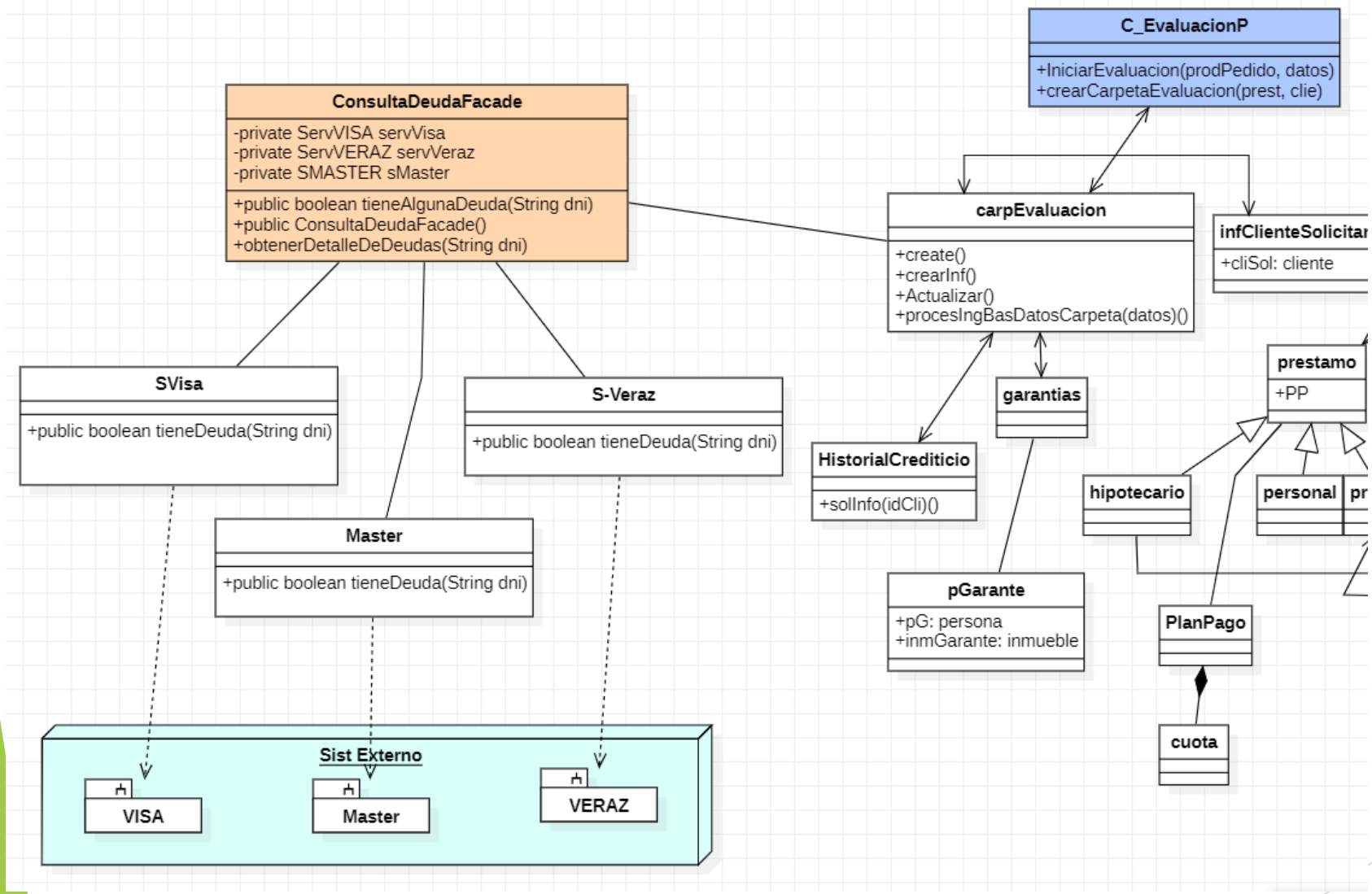
Facade::Conoce que clases subsistema son responsables de que peticiones y así, delega las peticiones a los objetos apropiados

Subsystem:Implementa la funcionalidad del subsistema, realiza el trabajo solicitado por el objeto Facade y no conoce, ni mantiene referencia alguna del objeto Facade



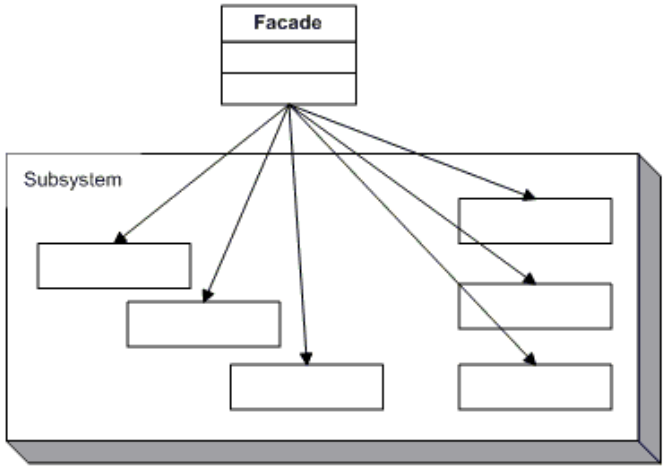
Placing orders by phone.

EJERCICIO: CASO BANCO OLIVARES



Las clases concretas, como sVISA, sVERAZ, etc., son las encargadas de conectarse directamente con el servicio externo (VISA, VERAZ, etc.), cada una representa a un servicio externo específico y conoce los detalles de cómo interactuar con el sist externo (por ejemplo, qué API llamar, qué formato usar, etc.).

El facade solo coordina las llamadas, pero no conoce ni debería conocer esos detalles



HACER: singleton, builder, prototype, abstract Factory, Factory Method, composite , bridge, proxy, visitor, chain of responsibility, iterator, template method, Visitor, adapter.

- ***Problema Específico de Diseño :****"DEFINIR EL PROBLEMA."*
- ***Categoría de Patrones? :****"DEFINIR LA CATEGORIA (CREACION/ESTRUCTURA/COMPORTAMIENTO)."*
- ***Solución, Modelarlo***
- ***grupo***
  1. singleton, chain of responsibility, composite
  2. builder, visitor, Decorator
  3. prototype, template method, proxy
  4. abstract Factory, facade
  5. Factory Method, singleton, visitor
  6. composite , adapter, obsercer
  7. bridge, iterator,facade